Clínica Virtual

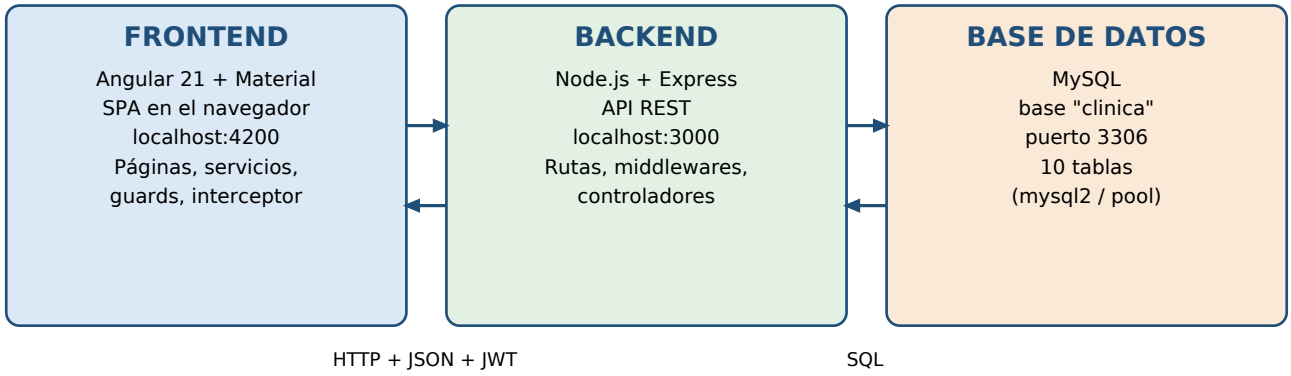
Flujo del frontend y del backend, organización del proyecto y arquitectura

Este documento explica cómo viaja una acción del usuario desde la pantalla hasta la base de datos y de vuelta, cómo está organizado cada repositorio y qué se guarda en cada carpeta del frontend.

Sección	Contenido
1. Arquitectura general	Las tres capas y cómo se comunican
2. Flujo del backend	Recorrido de una petición dentro de Express
3. Flujo del frontend	Arranque de Angular, navegación y llamadas a la API
4. Flujos completos	Login, solicitar turno y atender turno, de punta a punta
5. Organización del proyecto	Repositorios y estructura de carpetas
6. Carpetas del frontend	Qué se guarda en cada una
7. Arquitectura del backend	Carpetas, endpoints y tablas

1. Arquitectura general

El sistema sigue una arquitectura **cliente-servidor en tres capas**. Cada capa es independiente y se comunica solo con la vecina:



- **Presentación (frontend):** una SPA (Single Page Application) en Angular. Corre en el navegador, muestra las pantallas y nunca accede a la base de datos: todo lo pide a la API.
- **Lógica de negocio (backend):** una API REST en Node.js con Express. Valida datos, controla permisos por rol, aplica las reglas (horarios, superposición de turnos, etc.) y registra auditoría y notificaciones.
- **Datos (MySQL):** guarda usuarios, sedes, especialidades, coberturas, agendas, turnos, historial clínico, notificaciones y el log de auditoría.

Contrato entre frontend y backend

Todas las respuestas del backend, de éxito o de error, tienen la misma forma (función `responder()` en `utils/respuesta.js`):

```
{ "codigo": 201, "estado": "ok", "datos": { ...lo que se pidió... } }
{ "codigo": 409, "estado": "El horario solicitado ya está reservado...", "datos": null }
```

En el frontend esta forma está tipada como `RespuestaApi<T>`. El operador `datos()` extrae la parte útil y `mensajeDeError()` muestra el texto de `estado` cuando algo falla.

Autenticación con JWT

Al iniciar sesión el backend devuelve un **token JWT** firmado con `JWT_SECRET` que contiene `id`, `rol` e `id_sede` y vence según `JWT_EXPIRES_IN` (8 h). El frontend lo guarda en `localStorage` y lo envía en cada petición como `Authorization: Bearer <token>`. Hay cuatro roles: **paciente, médico, operador y admin**.

2. Flujo del backend

Cuando llega una petición HTTP, por ejemplo `POST /turnos`, recorre esta cadena:



Manejo de errores

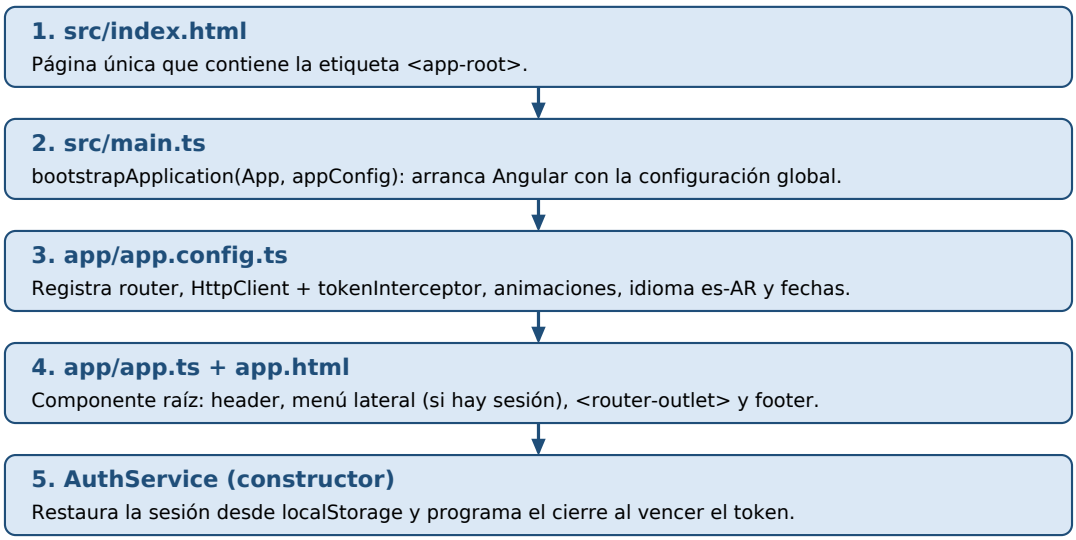
- Si ninguna ruta coincide, `app.js` responde **404 "Recurso no encontrado"**.
- Un manejador global atrapa JSON mal formado (400), cuerpos demasiado grandes y otros errores 4xx, y cualquier excepción no prevista como **500 "Error interno del servidor"**.
- Cada controlador tiene su propio `try/catch` y, cuando la operación modifica varias tablas, usa **transacciones** (`beginTransaction / commit / rollback`).

Servicios transversales

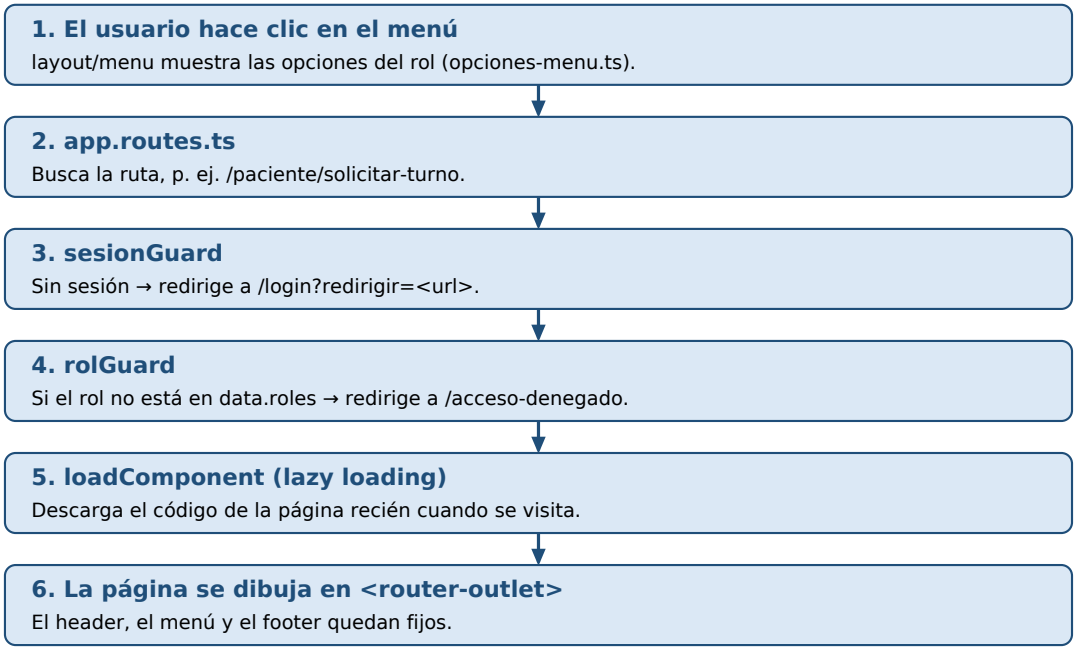
- **Auditoría** (`middlewares/auditoria.js`): se monta a nivel de router, así los controladores no necesitan saber que existe. Intercepta `res.json` y, si la respuesta fue 2xx, registra ALTA / MODIFICACION / BAJA con el usuario responsable y un detalle (sin la contraseña).
- **Notificaciones** (`utils/notificaciones.js`): cuando un turno se confirma, cancela o atiende, `notificarVarios()` inserta un aviso para el paciente, el médico y quien hizo la acción (sin duplicar).
- **Validaciones y fechas** (`utils/validaciones.js, utils/fechas.js`): formatos de DNI, email, teléfono, fecha AAAA-MM-DD, hora HH:MM y cálculo de los horarios de un rango de agenda.

3. Flujo del frontend

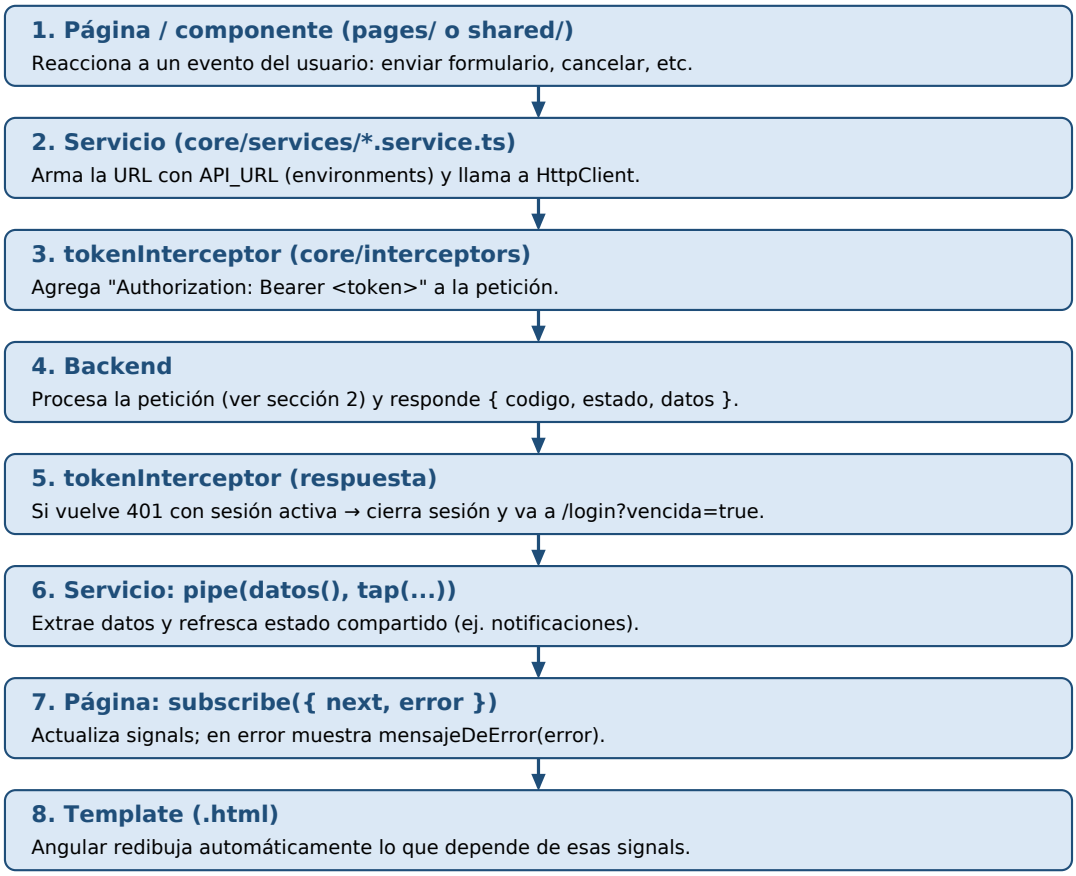
Arranque de la aplicación



Navegación entre pantallas



Llamada a la API desde una pantalla



Manejo del estado

El proyecto usa **signals** de Angular. Los servicios `providedIn: "root"` son únicos en toda la aplicación, así que funcionan como “almacén” compartido:

- `AuthService`: perfil, `estaLogueado`, `rol` y `nombreCompleto`; lo leen el header, el menú y los guards.
- `NotificacionService`: lista y contador `noLeidas` de la campana. `TurnoService` lo recarga después de crear, cancelar o atender un turno.
- Cada página tiene sus propias signals locales: `cargando`, `error`, listas, etc. Los componentes de `shared/` (`Cargando`, `Aviso`) muestran esos estados.

4. Flujos completos (de punta a punta)

4.1 Inicio de sesión

#	Dónde	Qué pasa
1	pages/login	El usuario ingresa DNI y contraseña; el formulario llama a AuthService.login().
2	AuthService	POST /auth/login con { dni, password }.
3	authController.login	Busca el usuario por DNI, compara la contraseña con bcrypt y firma el JWT (id, rol, id_sede).
4	AuthService	Guarda el token en localStorage y pide GET /auth/perfil (ya con el token del interceptor).
5	authController.perfil	Devuelve nombre, apellido, rol, cobertura, sede, etc.
6	AuthService	Guarda el perfil (signal + localStorage), programa el vencimiento y carga las notificaciones.
7	pages/login	Navega a inicioSegunRol() (/paciente, /medico, /operador o /admin) o a la URL "redirigir".

4.2 Solicitar un turno (paciente) / asignar un turno (operador)

#	Dónde	Qué pasa
1	pages/paciente/solicitar-turno	Recarga el perfil para tener la cobertura vigente y muestra el componente reutilizable.
2	shared/solicitud-turno	Stepper: especialidad → sede → médico → fecha → horario → nota. Cada paso filtra el siguiente.
3	CatalogoService / MedicoService	GET /especialidades, /sedes, /medicos y GET /disponibilidad/fechas y /horarios.
4	TurnoService.crear	POST /turnos con id_especialidad, id_sede, id_medico, fecha, hora, nota (+ id_paciente si es operador).
5	turnoRoutes	verificarToken + verificarRol("paciente","operador").
6	turnoController.crear	Valida formatos; el paciente solo reserva para sí y el operador solo en su sede; verifica que la hora caiga en un rango de la agenda.
7	turnoController.crear	Transacción: bloquea los turnos del médico (SELECT ... FOR UPDATE), rechaza con 409 si el horario está ocupado o el paciente ya tiene turno a esa hora, inserta el turno y las notificaciones, commit.
8	Respuesta	201 { codigo, estado: "ok", datos: turno }.
9	Frontend	TurnoService refresca la campana; la página muestra un aviso y navega a /paciente/turnos.

4.3 Atender un turno (médico)

#	Dónde	Qué pasa
1	pages/medico/turnos-programados	El médico ve sus turnos del día (GET /turnos/medico?fecha=...) y abre atender-dialogo.
2	TurnoService.atender	PUT /turnos/:id/atender con diagnóstico, tratamiento y observaciones.
3	turnoController.atender	Comprueba que el turno sea suyo y esté confirmado. Transacción: UPDATE turno → "atendido", INSERT en historial_clinico, notificaciones, commit.
4	Frontend	Se refresca la lista y la campana; el historial se ve en /medico/historial y /paciente/historial.

5. Organización del proyecto

El proyecto está dividido en **dos repositorios independientes**, que se clonan y se levantan por separado:

Repositorio	Tecnología	Comando	Puerto
frontend	Angular 21, Angular Material, Chart.js (ng2-charts), RxJS, TypeScript	npm start (ng serve)	4200
backend	Node.js, Express 4, mysql2, jsonwebtoken, bcrypt, cors, dotenv	npm run dev (nodemon)	3000

Raíz del repositorio frontend

```
frontend/
├── src/                    código fuente de la aplicación
│   ├── index.html          página única donde se monta
│   ├── main.ts             punto de entrada: arranca Angular
│   ├── styles.scss         estilos globales y tema de Material
│   ├── environments/       configuración por entorno (URL de la API)
│   └── app/                toda la aplicación (ver sección 6)
├── public/                 archivos estáticos copiados tal cual (favicon.ico)
├── docs/                   guía de instalación y guías en PDF
├── .vscode/                tareas, depuración y extensiones recomendadas
├── angular.json            configuración del build y del servidor de desarrollo
├── package.json            dependencias y scripts (start, build, test)
├── tsconfig*.json         configuración de TypeScript (app y tests)
└── .prettierrc, .editorconfig   formato del código
```

Raíz del repositorio backend

```
backend/
├── src/
│   ├── server.js           levanta el servidor HTTP
│   ├── app.js              configura Express, 404 y errores globales
│   ├── routes/             una ruta por recurso + index.js
│   ├── middlewares/        auth.js (JWT y roles), auditoria.js
│   ├── controllers/        lógica de cada recurso
│   ├── database/db.js      pool de conexiones a MySQL
│   └── utils/              respuesta, validaciones, fechas, notificaciones
├── database/               script SQL con la estructura y datos de ejemplo
├── scripts/                importar-db.js / .ps1 (carga la base)
├── postman/                colección para probar la API
├── .env.example            variables: PORT, DB_*, JWT_SECRET, JWT_EXPIRES_IN
└── package.json            dependencias y scripts (dev, start, db:importar)
```

6. Carpetas del frontend: qué se guarda en cada una

Dentro de `src/app/` el código está separado por responsabilidad: **core** (lo que existe una sola vez y no se dibuja), **layout** (el marco fijo), **shared** (piezas reutilizables) y **pages** (las pantallas).

```
src/app/
├── app.ts / app.html / app.scss   componente raíz
├── app.config.ts                  providers globales
├── app.routes.ts                  mapa de rutas con guards
├── core/
│   ├── models/          interfaces TypeScript
│   ├── services/        comunicación con la API y estado
│   ├── guards/          sesion.guard.ts, rol.guard.ts
│   └── interceptors/    token.interceptor.ts
├── layout/              header/, menu/, footer/
├── shared/
│   ├── components/      componentes reutilizables
│   ├── pipes/           fecha.pipe.ts
│   └── utils/           fechas, adaptador de fecha, paginador
└── pages/               una carpeta por pantalla, agrupadas por rol
```

Carpeta	Qué se guarda	Ejemplos
src/environments/	Constantes que cambian según el entorno. Es el único lugar donde se define la URL de la API.	environment.ts, environment.development.ts (apiUrl: http://localhost:3000)
app/ (raíz)	El componente raíz, la configuración global de la app y la tabla de rutas.	app.ts, app.config.ts, app.routes.ts
core/models/	Interfaces y tipos TypeScript que describen los datos que van y vienen de la API. No tienen lógica.	turno.model.ts (Turno, NuevoTurno, EstadoTurno), usuario.model.ts (Perfil, Rol), respuesta.model.ts (RespuestaApi)
core/services/	Servicios inyectables (uno por recurso del backend) que hacen las peticiones HTTP y guardan estado compartido. Además, utilidades comunes de la API y de errores.	auth, turno, agenda, paciente, medico, catalogo, usuario, notificacion, auditoria, reporte; api.ts (API_URL, datos(), parametros()); errores.ts
core/guards/	Funciones que deciden si se puede entrar a una ruta.	sesionGuard (requiere sesión), rolGuard (requiere el rol de data.roles)
core/interceptors/	Código que se ejecuta en todas las peticiones HTTP.	tokenInterceptor: agrega el JWT y maneja el 401
layout/	Componentes del marco visual que están siempre en pantalla.	header (título, campana de notificaciones, menú de usuario), menu (opciones-menu.ts por rol), footer
shared/components/	Componentes reutilizados por varias pantallas, sin ruta propia. Reciben datos por input() y avisan por output().	solicitud-turno, formulario-paciente, buscador-pacientes, lista-historial, estado-turno, confirmar-dialogo, rango-dialogo, aviso, cargando
shared/pipes/	Pipes para transformar valores en los templates.	fecha.pipe.ts (AAAA-MM-DD → dd/mm/aaaa)
shared/utils/	Funciones auxiliares y adaptadores de Angular Material.	fechas.ts, adaptador-fecha.ts (datepicker en es-AR), paginador-es.ts (textos del paginador)
pages/	Las pantallas: cada una es un componente cargado por una ruta. Se agrupan en subcarpetas por rol.	ver tabla siguiente
public/	Archivos estáticos que se sirven sin procesar.	favicon.ico
docs/	Documentación del proyecto.	GUIA-INSTALACION.md/.pdf, guías de flujos

Estructura de cada componente

Cada componente vive en su carpeta con hasta tres archivos del mismo nombre: `.ts` (lógica, signals e inyección de servicios), `.html` (template) y `.scss` (estilos solo de ese componente). Los más chicos tienen el template dentro del `.ts`. Los diálogos (`*-dialogo.ts`) se guardan junto a la página que los abre. Todos son **standalone**: declaran sus propios `imports`.

Contenido de pages/

Subcarpeta	Ruta	Pantallas
inicio, login, registro	/, /login, /registro	Portada pública, inicio de sesión y registro de pacientes.
perfil	/perfil	Datos del usuario logueado (cualquier rol).
paciente/	/paciente/*	mis-turnos, solicitar-turno, mi-historial, mis-datos.
medico/	/medico/*	mi-agenda (rangos horarios), turnos-programados (+ atender-dialogo), historial-clinico.
operador/	/operador/*	agenda-sede (+ diálogos de agenda y turnos del médico), alta-paciente, asignar-turno.
admin/	/admin/*	abm-catalogo (un único ABM genérico para sedes, especialidades y coberturas, configurado desde app.routes.ts), usuarios (+ usuario-dialogo), auditoria, reportes (gráficos con Chart.js).
acceso-denegado, no-encontrada	/acceso-denegado, **	Páginas de error: rol sin permiso y URL inexistente.

7. Arquitectura del backend

El backend usa una arquitectura **en capas tipo MVC sin vistas** (Rutas → Middlewares → Controladores → Base de datos). Cada recurso tiene su par `xxxRoutes.js` / `xxxController.js`. No hay ORM: los controladores escriben SQL parametrizado sobre el pool de `mysql2`.

Carpeta	Responsabilidad
routes/	Qué URL y método existen y qué middlewares protegen cada uno (token, rol, auditoría). No contiene lógica.
middlewares/	Código que corre antes del controlador: autenticación JWT, control de roles, token opcional y auditoría automática.
controllers/	Lógica de negocio: validaciones, permisos finos (ej. operador solo en su sede), consultas SQL, transacciones y respuesta.
database/db.js	Pool único de conexiones a MySQL configurado desde .env (dateStrings: true para no desplazar fechas).
utils/	Funciones compartidas: respuesta uniforme, validaciones de datos personales, manejo de fechas/horarios y notificaciones.

Endpoints principales

Prefijo	Endpoints	Roles
/auth	POST /registro, POST /login, GET y PUT /perfil	Público / logueado / paciente (PUT)
/turnos	POST /, GET /mios, GET /medico, GET /sede, PUT /:id/cancelar, PUT /:id/atender	paciente, operador, médico
/agenda	CRUD de rangos horarios del médico	médico, operador
/disponibilidad	GET /fechas, GET /horarios	paciente, operador
/historial	POST /, GET /:idPaciente	médico, paciente
/sedes, /especialidades, /coberturas	GET (listar) y POST/PUT/DELETE	listar: logueado (coberturas público); ABM: admin
/medicos, /pacientes	GET (listar / buscar)	logueado / operador, médico, admin
/usuarios	GET, GET /:id, POST, PUT /:id	admin
/notificaciones	GET /, PUT /:id/leida	logueado
/auditoria, /reportes	Log de cambios y 4 reportes estadísticos	admin
/health	GET / (estado del servidor)	público

Modelo de datos (MySQL)

Tabla	Qué guarda
usuario	Todos los usuarios con su rol (paciente, medico, operador, admin), contraseña con hash bcrypt, sede y cobertura.
sede, especialidad, cobertura	Catálogos administrados por el admin.
medico_especialidad	Relación muchos a muchos entre médicos y especialidades.
agenda	Rangos de atención: médico, especialidad, sede, fecha, hora de entrada y salida.
turno	Reserva dentro de una agenda: fecha, hora, paciente, cobertura, nota y estado (confirmado, cancelado, atendido).
historial_clinico	Diagnóstico, tratamiento y observaciones de cada turno atendido.
notificacion	Avisos por usuario con su estado leída / no leída.
log_auditoria	Quién hizo qué alta, baja o modificación, sobre qué entidad y cuándo.

Resumen

El **frontend** decide qué se muestra y protege la navegación (guards), pero la seguridad real está en el **backend**, que vuelve a verificar token y rol en cada petición y aplica todas las reglas de negocio antes de tocar la **base de datos**. Ambos lados se entienden gracias al formato único de respuesta y al token JWT.